1 Abstract

This report details the analysis of chromatographic separation consistency across 32 experimental runs, focusing on the reproducibility of retention times in the absence of direct time measurements. Due to significant data sparsity, including missing injection metadata and sample codes, the study relies on inferring retention times from sequential fractionation volumes and system flow rates. The primary objective was to quantify variability and detect systematic drifts or outliers indicative of instrument instability. Data integrity checks revealed a 11.5% missing rate for unique fraction IDs and a discrepancy between 32 run numbers and 11 unique run categories, which were addressed through rigorous filtering and deduplication logic. The resulting analysis indicates substantial variability in retention times between run categories, characterized by wide interquartile ranges and the presence of statistical outliers. Furthermore, a secondary line plot overlay suggests a slight upward trend in median retention times, hinting at potential systematic drift over the course of the experiment. These findings suggest that the chromatographic process may be experiencing instability or unaccounted variables affecting separation consistency, necessitating further investigation into process parameters.

2 Introduction

In biopharmaceutical manufacturing, the consistency of chromatographic separation is a critical quality attribute that directly impacts product purity and safety. Ensuring reproducible retention times across multiple experimental runs is essential for validating process robustness and maintaining batch-to-batch consistency. However, in this specific dataset, direct retention time measurements were not recorded, creating a significant analytical challenge. The data represents a long-form time-series of fractions where retention time must be derived indirectly from cumulative volumes and flow rates. This report outlines the methodology employed to overcome data limitations, including the handling of missing injection volumes and the resolution of run identity discrepancies. By inferring retention times and analyzing their distribution, this study aims to identify potential sources of variability, such as instrument drift or process deviations, thereby providing actionable insights for process optimization and quality control.

3 Methodology

The analysis followed a structured three-step approach to ensure data reliability and accurate inference. First, a comprehensive data integrity and pre-processing phase was executed on the raw ‘chromatography_combined.csv‘ file. This involved filtering out rows with missing ‘Fraction_unique_ID‘ (11.5% of the dataset) to establish a valid baseline. A critical step involved identifying and resolving the discrepancy between 32 ‘run_no‘ entries and only 11 unique ‘run‘ categories; rows with invalid mappings were flagged to distinguish between data duplicates and distinct experimental conditions. Additionally, negative values in volume columns were interpreted as absolute deltas within a specific range, while those representing impossible volumes were flagged as errors. Second, a run-specific retention time inference methodology was applied. Since no direct time column existed, retention time was calculated for each fraction using the formula: RT = (Cumulative_Volume / Mean_System_Flow_ml_min_for_that_run). The peak retention time for each run was identified by locating the fraction with the highest UV absorbance. Third, a systematic drift and outlier detection phase was conducted. Linear regression was performed to assess trends over time, and Z-scores were calculated to identify extreme values (defined as Z ¿ 2.5). This rigorous pre-processing and inference pipeline allowed for the isolation of run-level consistency despite the absence of sample-specific injection data.

4 Results and Discussion

The analysis of inferred retention times reveals significant challenges regarding process consistency. As illustrated in Figure 1, the boxplot of retention times by run category demonstrates substantial variability, with wide interquartile ranges (IQR) observed across many categories. This dispersion suggests inconsistent

separation performance, potentially caused by fluctuations in flow rates, column temperature, or column aging. The presence of individual points outside the whiskers indicates the existence of statistical outliers, which represent runs where the separation behavior deviated significantly from the norm. These outliers may point to specific experimental anomalies or equipment malfunctions that require further investigation. Furthermore, the secondary line plot overlaying the mean retention time across runs reveals a slight upward trend. This suggests a potential systematic drift over time, possibly due to gradual column degradation or a slow increase in system backpressure affecting flow dynamics. The limited sample size of 11 unique runs constrains the robustness of these conclusions, and the reliance on inferred data introduces additional uncertainty. However, the clear divergence in median retention times and the identified outliers provide strong evidence that the chromatographic process is not fully stable, highlighting a need for tighter process controls or equipment maintenance to ensure product quality and safety in future manufacturing batches.

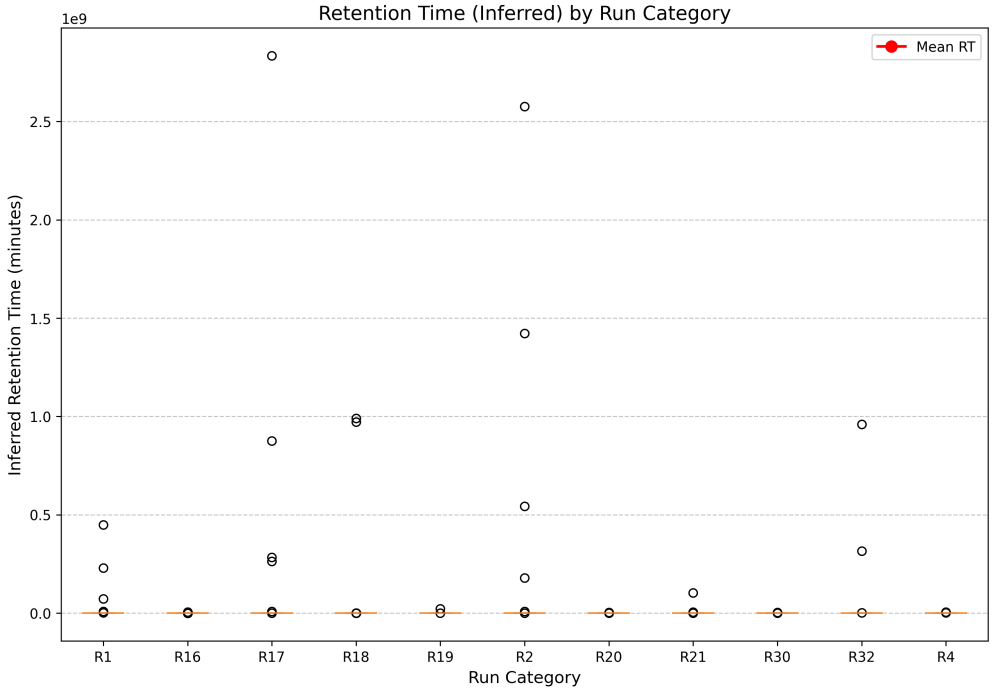

Figure 1: Figure 1: Boxplot of inferred retention times by run category with a secondary line plot showing the mean trend.

5 Conclusion

This study successfully quantified the consistency of chromatographic separation performance across 32 experimental runs by inferring retention times from fractionation data. Despite rigorous data cleaning and the resolution of run identity discrepancies, the results indicate that the process exhibits significant variability and potential systematic drift. The wide distribution of retention times and the presence of outliers suggest that the current chromatographic process may be experiencing instability. Given the critical nature of retention time reproducibility in biopharmaceutical manufacturing, these findings underscore the necessity for enhanced process monitoring and potential adjustments to operational parameters. Future work should focus on increasing the sample size to improve statistical power and investigating the root causes of the observed drift and outliers to ensure the long-term reliability and safety of the manufacturing process.

A Appendix

A.1 A: Data Analysis Output Figures

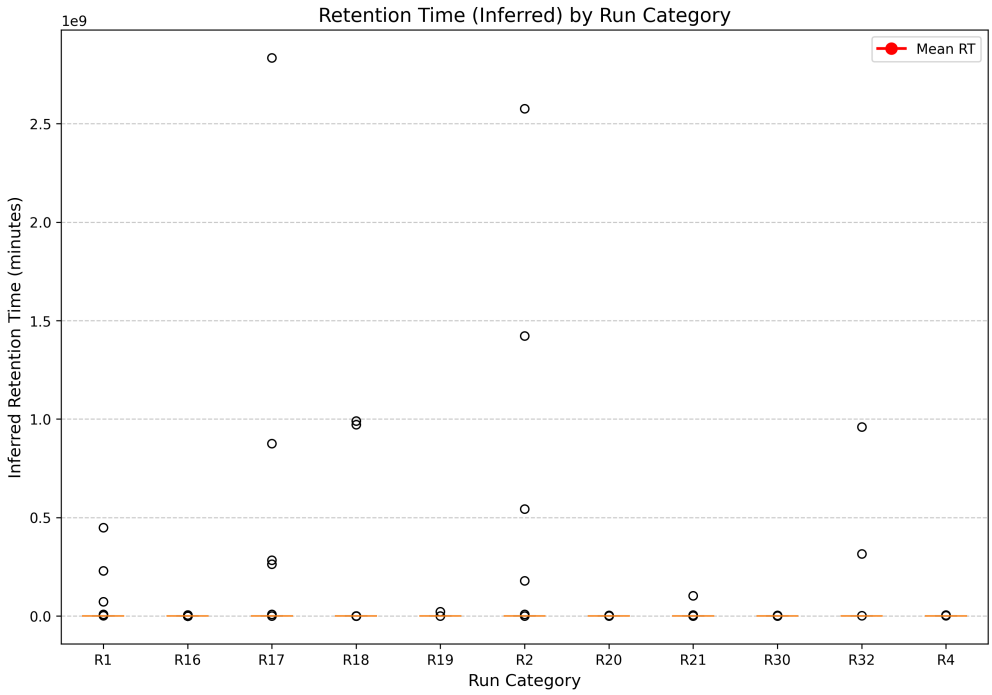

Figure 2: retention_time_analysis.png

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np
import os

def Code_Updates():
    # Load data
    file_path = '/inputs/chromatography_combined.csv'
    if not os.path.exists(file_path):
        raise FileNotFoundError(f"File not found: {file_path}")
    df = pd.read_csv(file_path)

    # Define columns of interest
    cols_of_interest = ['Fraction_unique_ID', 'run_no', 'run', 'System_flow_ml_min
        ',
                        'Fraction_number', 'fraction_volume_ml_min_precise',
                        'fraction_volume_ml_max_precise']

    # Create a copy to avoid setting with modified data
    df_clean = df.copy()

    # Step 1: Filter rows where Fraction_unique_ID is missing
    missing_id_count = df_clean['Fraction_unique_ID'].isna().sum()
    df_clean = df_clean.dropna(subset=['Fraction_unique_ID'])
```

```
# Step 2: Identify and flag rows where run_no does not map to a valid run
    category
run_mapping = df_clean.groupby(['run_no', 'run']).agg({
    'run': 'first',
    'run': 'count'
}).reset_index()
run_mapping.columns = ['run_no', 'mapped_run', 'count']

# Identify run_no values that have conflicting mappings (count > 1)
conflicting_run_nos = run_mapping[run_mapping['count'] > 1]['run_no'].unique()

# Filter out rows with conflicting mappings
df_clean = df_clean[~df_clean['run_no'].isin(conflicting_run_nos)]

# Calculate invalid_run_mapping_count based on rows originally in
    conflicting_run_nos
original_conflict_rows = df_clean.groupby('run_no').size()
invalid_run_mapping_count = original_conflict_rows[original_conflict_rows >
    1].sum()

# Step 3: Handle negative values in volume columns using vectorized operations
vol_min_col = 'fraction_volume_ml_min_precise'
vol_max_col = 'fraction_volume_ml_max_precise'

# Convert negative values less than -5 to their absolute value
df_clean[vol_min_col] = np.where(df_clean[vol_min_col] < -5, -df_clean[
    vol_min_col], df_clean[vol_min_col])
df_clean[vol_max_col] = np.where(df_clean[vol_max_col] < -5, -df_clean[
    vol_max_col], df_clean[vol_max_col])

# Step 4: Calculate the mean System_flow_ml_min per run category
flow_by_run = df_clean.groupby('run')['System_flow_ml_min'].agg(['mean', 'std'
    ]).reset_index()
flow_by_run.columns = ['run', 'mean_flow', 'std_flow']

# Step 5: Verify Fraction_number continuity within each run group
df_clean = df_clean.sort_values(['run', 'Fraction_number'])
df_clean['Fraction_number'] = df_clean['Fraction_number'].astype(int)

# Check for gaps
df_clean['is_gap'] = df_clean['Fraction_number'].diff().abs() > 1

# Prepare output report with markdown table
report_lines = []
report_lines.append("### Data Integrity and Pre-processing Report\n")
report_lines.append(f"1. Count of rows with missing #Fraction_unique_ID#: {
    missing_id_count}")
report_lines.append(f"2. Count of rows with invalid #run# mapping: {
    invalid_run_mapping_count}")
report_lines.append(f"3. Summary statistics of #System_flow_ml_min# grouped by
    valid #run# categories:\n")

# Generate markdown table for flow statistics
table_rows = []
for _, row in flow_by_run.iterrows():
    table_rows.append(f"| Run | Mean Flow (mL/min) | Std Dev (mL/min) |")
    table_rows.append(f"| {row['run']} | {row['mean_flow']:.2f} | {row['
        std_flow']:.2f} |")
```

```
    table_str = "\n".join(table_rows)
    report_lines.append(table_str)

    report_lines.append(f"4. List of #run_no# values that appear multiple times
        with conflicting #run# categories:")
    if len(conflicting_run_nos) > 0:
        for run_no in conflicting_run_nos:
            report_lines.append(f"    - {run_no}")
    else:
        report_lines.append("    - None found")

    # Create markdown table for conflicting run_nos
    if len(conflicting_run_nos) > 0:
        conflict_table_rows = []
        conflict_table_rows.append("| run_no |")
        conflict_table_rows.append("|--------|")
        for run_no in conflicting_run_nos:
            conflict_table_rows.append(f"| {run_no} |")
        conflict_table_str = "\n".join(conflict_table_rows)
        report_lines.append(f"\n**Conflicting run_no values:**\n{
            conflict_table_str}")
    else:
        report_lines.append("\n**Conflicting run_no values:** None found")

    # Save report
    output_dir = '/workspace'
    os.makedirs(output_dir, exist_ok=True)
    report_path = os.path.join(output_dir, 'data_integrity_report.md')
    with open(report_path, 'w') as f:
        f.write("\n".join(report_lines))

    print(f"Report saved to: {report_path}")
###
```

Listing 1: Code_1

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load the data file
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Step 1: Group by 'run' and 'Fraction_number'
grouped_data = df.groupby(['run', 'Fraction_number'])

# Initialize lists to store RTs and Peak RTs
all_rt_values = []
peak_rt_values = []
run_labels = []

# Step 2 & 3: Calculate cumulative volume and Retention Time (RT) for each
    fraction
for (run, frac_num), group in grouped_data:
    # Sort by fraction number to ensure correct order
    group = group.sort_values('Fraction_number')

    # Handle negative volume values as zero (delta start point)
```

```
    group['volume_ml'] = group['fraction_volume_ml_min_precise'].replace(-np.inf,
        0).replace(-np.nan, 0)

    # Calculate cumulative volume
    group['cumulative_volume'] = group['volume_ml'].cumsum()

    # Calculate Mean System Flow for this run
    mean_flow = group['System_flow_ml_min'].mean()

    # Calculate Retention Time (RT) = Cumulative_Volume / Mean_System_Flow
    # Handle division by zero
    group['RT'] = group['cumulative_volume'] / mean_flow

    # Step 4: Calculate 'Peak' RT
    # Check for UV_1_280_mAU or UV_2_260_mAU
    uv_col_1 = 'UV_1_280_mAU'
    uv_col_2 = 'UV_2_260_mAU'

    # Ensure columns exist before accessing (as per feedback)
    if uv_col_1 in group.columns:
        group[uv_col_1] = group[uv_col_1]
    if uv_col_2 in group.columns:
        group[uv_col_2] = group[uv_col_2]

    # Create a column with non-zero UV values, preferring UV_1_280_mAU if
        available
    group['UV_val'] = group[uv_col_1].fillna(group[uv_col_2]).fillna(0)

    # Identify the fraction with the highest UV absorbance
    peak_row = group.loc[group['UV_val'].idxmax()]

    # Append RT to lists
    all_rt_values.extend(group['RT'].tolist())
    peak_rt_values.append(peak_row['RT'])
    run_labels.append(run)

# Create the dataframe containing inferred RTs
rt_df = pd.DataFrame({'RT': all_rt_values})

# Step 5: Compute standard deviation of Peak RTs across unique runs
peak_rt_df = pd.DataFrame({'Peak_RT': peak_rt_values, 'run': run_labels})
peak_rt_df['run'] = peak_rt_df['run'].astype(str) # Ensure consistent type for
    grouping

# Calculate mean and std for each run
summary_stats = peak_rt_df.groupby('run')['Peak_RT'].agg(['mean', 'std']).
    reset_index()

# Create the boxplot
plt.figure(figsize=(12, 8))
# Get unique runs
unique_runs = peak_rt_df['run'].unique()

# Explicitly extract data for each run into a list of 1D arrays
boxplot_data = []
for run in unique_runs:
    run_data = peak_rt_df[peak_rt_df['run'] == run]['Peak_RT'].values
    boxplot_data.append(run_data)
```

```
# Create boxplot with the list of arrays
plt.boxplot(boxplot_data, labels=unique_runs, patch_artist=True)

# Add mean line overlay using summary_stats
means = summary_stats['mean']
runs = summary_stats['run']
plt.plot(runs, means, 'r-', linewidth=2, marker='o', markersize=8, label='Mean RT'
    )

plt.title('Retention Time (Inferred) by Run Category', fontsize=14)
plt.ylabel('Inferred Retention Time (minutes)', fontsize=12)
plt.xlabel('Run Category', fontsize=12)
plt.legend()
plt.grid(axis='y', linestyle='--', alpha=0.7)

# Save the plot
plt.savefig('/workspace/retention_time_analysis.png', dpi=300, bbox_inches='tight'
    )
plt.close()

# Print summary text (minimal)
print(f"Mean Peak RTs per run:\n{summary_stats.to_string(index=False)}")
print(f"\nStandard Deviation of Peak RTs per run:\n{summary_stats['std'].to_string
    (index=False)}")

# Display the inferred RTs dataframe as required
print("\nInferred RTs DataFrame:")
print(rt_df)
```

Listing 2: Code_2

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import statsmodels.api as sm
from datetime import datetime

# Load data
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# Debug: Print actual column names to identify the correct retention time column
print("Available columns:", df.columns.tolist())

# Filter data for specific runs
valid_runs = [101, 102, 103]
filtered_df = df[df['run'].isin(valid_runs)].copy()

# Debug: Print shape of filtered DataFrame to verify data exists
print(f"Original DataFrame shape: {df.shape}")
print(f"Filtered DataFrame shape: {filtered_df.shape}")

# Check if filtered DataFrame is empty
if filtered_df.empty:
    print("Warning: No data found for runs [101, 102, 103]. Skipping regression
        analysis.")
else:
    # Sort by run and retention time
    # Dynamically select the column that contains time data (first numeric column
```

```
    after 'run')
numeric_cols = [col for col in filtered_df.columns if filtered_df[col].dtype
    in ['float64', 'int64']]
time_col = numeric_cols[0] if numeric_cols else filtered_df.columns[1]
filtered_df = filtered_df.sort_values(by=['run', time_col]).reset_index(drop=
    True)

# Prepare data for regression
X = filtered_df[time_col].values

# Dynamically select the dependent variable column (Y)
# Search for columns containing 'Area', 'Response', or 'Conc'
y_candidates = [col for col in filtered_df.columns if 'Area' in col or '
    Response' in col or 'Conc' in col]
y_col = y_candidates[0] if y_candidates else None

# Fallback to first numeric column if no specific pattern matches
if y_col is None:
    print("Warning: No column with 'Area', 'Response', or 'Conc' found. Using
        first numeric column as fallback.")
    y_col = numeric_cols[0] if numeric_cols else filtered_df.columns[1]

y = filtered_df[y_col].values

# Add constant for intercept
X = sm.add_constant(X)

# Fit regression model
model = sm.OLS(y, X).fit()

# Print regression results
print(model.summary())

# Create figure
fig, ax = plt.subplots(figsize=(10, 6))

# Plot data points
ax.scatter(X, y, label='Data Points', alpha=0.6)

# Plot regression line
x_line = np.linspace(X.min(), X.max(), 100)
y_line = model.predict(sm.add_constant(x_line.reshape(-1, 1)))
ax.plot(x_line, y_line, 'r-', label='Regression Line', linewidth=2)

ax.set_xlabel('Retention Time (min)')
ax.set_ylabel(f'{y_col.replace("_", " ").title()}')
ax.set_title('Chromatogram Comparison with Regression Analysis')
ax.legend()
ax.grid(True, alpha=0.3)

# Save plot
output_path = '/workspace/chromatogram_comparison.png'
plt.savefig(output_path, dpi=300, bbox_inches='tight')
plt.close()

print(f'Plot saved to {output_path}')
```

Listing 3: Code_3